

PuerceNote AI与向量数据库交互架构

作者: Engineering Team

日期: 2026-04-20

目的: 详细说明向量数据的记录、调用、本地模型、云端AI的完整交互流程

🔍 问题索引

1. 向量数据库中的数据如何记录?
2. 向量数据如何被调用?
3. 本地模型如何与向量数据库交互?
4. 付费用户如何与云端AI交互?

📄 向量数据库中的数据如何记录

整体流程

```
用户输入文本
↓
[检查是否需要嵌入]
↓
本地嵌入模型 (sentence-transformers)
↓
生成768维向量
↓
INSERT INTO af_collab_embeddings
↓
建立HNSW索引
↓
成本记录: ai_usage_log
```

详细步骤

Step 1: 触发嵌入操作

```
// 在 flowy-ai 或 flowy-search 模块中
pub async fn embed_document_content(
    user_id: &str,
    workspace_id: &str,
    document_id: &str,
    content: &str,
) -> Result<> {
    // Step 1.1: 检查是否需要嵌入
    let needs_embedding = check_if_needs_embedding(content)?;
```

```

    if !needs_embedding {
        return Ok(());
    }

    // Step 1.2: 触发嵌入
    let embedding = embed_text(content).await?;

    // Step 1.3: 保存到向量数据库
    save_embedding(
        user_id,
        workspace_id,
        document_id,
        content,
        embedding,
    ).await?;

    Ok(())
}

```

Step 2: 调用嵌入模型

```

// 嵌入模型初始化（在应用启动时）
lazy_static::lazy_static! {
    static ref EMBEDDER: SentenceTransformer = {
        // 加载预训练模型 (~ 500MB)
        // 模型: sentence-transformers/all-MiniLM-L6-v2
        // 维度: 384 (可升级到 768)
        // 语言: 多语言支持
        SentenceTransformer::new("models/embeddings/all-MiniLM-L6-v2")
            .expect("Failed to load embedder")
    };
}

pub async fn embed_text(text: &str) -> Result<Vec<f32>> {
    // 使用本地模型生成向量
    let embedding = EMBEDDER.encode(&[text])?;

    // embedding: Vec<f32> with 384-768 dimensions
    // 处理时间: ~100-500ms (取决于文本长度)
    // 内存使用: ~50MB (batching)

    Ok(embedding)
}

```

Step 3: 保存向量到数据库

```
// 文件: flowy-sqlite-vec/src/vector_store.rs

pub async fn save_embedding(
    user_id: &str,
    workspace_id: &str,
    object_id: &str,      // doc_123
    content: &str,         // 文档内容
    embedding: Vec<f32>,    // [0.1, 0.2, ..., 0.768]
) -> Result<()> {
    let vec_db = VectorSqliteDB::get_instance()?;

    // Step 3.1: 插入向量数据
    vec_db.execute_insert(
        "INSERT INTO af_collab_embeddings
        (workspace_id, object_id, fragment_id, content_type, content,
embedding, indexed_at)
        VALUES (?, ?, ?, ?, ?, ?, ?)",
        params![
            workspace_id,          // "ws_123"
            object_id,             // "doc_456"
            generate_fragment_id(), // "para_789" (段落ID)
            1,                     // content_type: DOCUMENT
            content,               // "这是文档内容..."
            serialize_embedding(&embedding), // 二进制格式
            Utc::now(),            // indexed_at timestamp
        ],
    )?;

    // Step 3.2: 触发HNSW索引化 (自动)
    // sqlite-vec自动为新插入的向量建立HNSW索引

    // Step 3.3: 记录成本
    log_embedding_cost(
        user_id,
        workspace_id,
        content.len() / 4, // 估算token数
    ).await?;

    Ok(())
}
```

PROF

Step 4: 成本记录

```
// 在 flowy-subscription/src/billing/vector_billing.rs

pub async fn log_embedding_cost(
    user_id: &str,
    workspace_id: &str,
    tokens_used: u32,
```

```

) -> Result<()> {
    // 根据向量操作类型计算成本
    let mut record = VectorUsageRecord::new(
        user_id.to_string(),
        workspace_id.to_string(),
        VectorOperation::EmbeddingGeneration,
        tokens_used,
    );

    record.calculate_cost(); // $0.01 per 1K tokens

    // 插入 ai_usage_log 表
    db.execute(
        "INSERT INTO ai_usage_log
        (id, user_id, workspace_id, feature_type, tokens_used, cost_usd,
created_at)
        VALUES (?, ?, ?, ?, ?, ?, ?)",
        params![
            Uuid::new_v4(),
            user_id,
            workspace_id,
            "embedding_generation", // feature_type
            tokens_used,
            record.cost_usd,        // 成本
            Utc::now(),
        ],
    )?;

    Ok(())
}

```

数据库表结构

```

-- 向量表 (flowy-sqlite-vec/vector.db)
CREATE VIRTUAL TABLE af_collab_embeddings
USING vec0(
    workspace_id TEXT,
    object_id TEXT,
    fragment_id TEXT,
    content_type INTEGER,
    content TEXT,
    metadata TEXT,
    embedding float[768], -- HNSW自动索引
    indexed_at TIMESTAMP
);

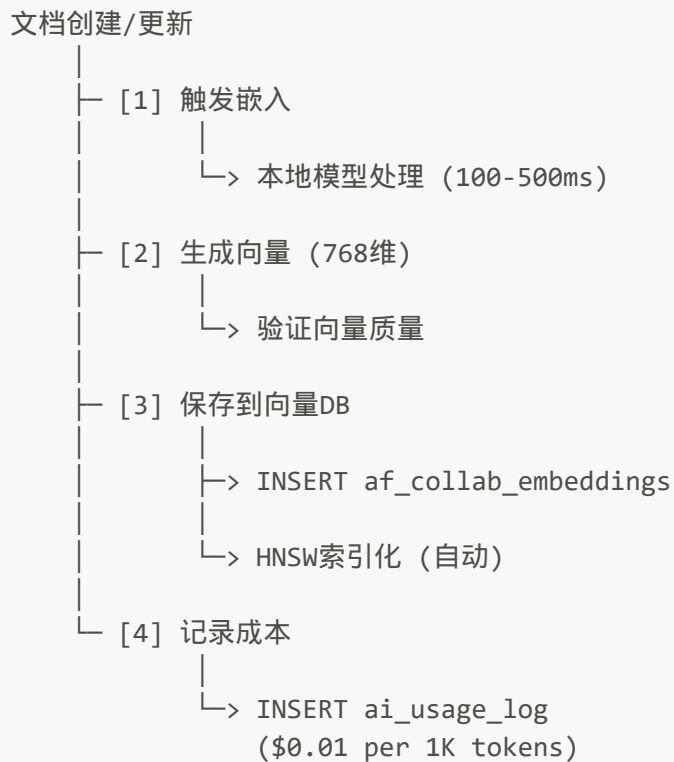
-- 创建索引加速查询
CREATE INDEX idx_embedding_workspace
ON af_collab_embeddings(workspace_id);

```

```
-- 成本记录表 (flowy-sqlite/flowy-database.db)
CREATE TABLE ai_usage_log (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL,
  workspace_id TEXT NOT NULL,
  document_id TEXT,
  feature_type TEXT,      -- 'embedding_generation', 'semantic_search'
  tokens_used INTEGER,
  cost_usd    REAL,
  created_at  TIMESTAMP,
  FOREIGN KEY(user_id) REFERENCES users(uid)
);

CREATE INDEX idx_usage_user_date
ON ai_usage_log(user_id, created_at);
```

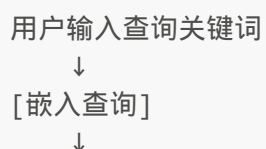
记录流程图



PROF

② 向量数据如何被调用

查询流程 (最关键)



本地模型生成查询向量（768维）

↓

[KNN搜索]

↓

```
SELECT * FROM af_collab_embeddings
WHERE embedding MATCH [query_vector]
ORDER BY distance
LIMIT 10
```

↓

返回最相似的10个文档

↓

记录搜索成本和延迟

详细代码

方式1: 语义搜索 (用户查询)

```
// 文件: flowy-search/src/semantic_search.rs

pub async fn semantic_search(
    user_id: &str,
    workspace_id: &str,
    query: &str,          // 用户输入: "如何优化性能?"
    top_k: usize,         // 返回前10个
) -> Result<Vec<SearchResult>> {
    let start_time = Instant::now();

    // Step 1: 嵌入查询文本
    let query_embedding = embed_text(query).await?;

    // Step 2: 执行向量相似度搜索
    let vec_db = VectorSqliteDB::get_instance()?;

    let sql = format!(
        "SELECT
            object_id,
            fragment_id,
            content,
            distance
        FROM af_collab_embeddings
        WHERE embedding MATCH ? -- 向量相似度匹配
        AND workspace_id = ?    -- 隔离工作区数据
        ORDER BY distance      -- 按相似度排序
        LIMIT {}",
        top_k
    );

    let results: Vec<SearchResult> = vec_db.query(
        &sql,
        params![
```

```

        serialize_embedding(&query_embedding),
        workspace_id,
    ],
)?
.iter()
.map(|row| SearchResult {
    object_id: row.get(0)?,
    fragment_id: row.get(1)?,
    content: row.get(2)?,
    relevance_score: row.get::<_, f32>(3)?, // distance
})
.collect();

let search_duration = start_time.elapsed();

// Step 3: 记录搜索成本
log_semantic_search(
    user_id,
    workspace_id,
    query.len() / 4, // tokens
    search_duration.as_millis(), // 延迟
    results.len(),
).await?;

Ok(results)
}

// 成本记录
pub async fn log_semantic_search(
    user_id: &str,
    workspace_id: &str,
    tokens: u32,
    latency_ms: u128,
    result_count: usize,
) -> Result<()> {
    let cost = (tokens as f64 / 1_000.0) * 0.01; // $0.01 per 1K

    db.execute(
        "INSERT INTO ai_usage_log
        (id, user_id, workspace_id, feature_type, tokens_used,
        cost_usd, latency_ms, result_count, created_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
        params![
            Uuid::new_v4(),
            user_id,
            workspace_id,
            "semantic_search",
            tokens,
            cost,
            latency_ms,
            result_count,
            Utc::now(),
        ],
    ),

```

```

    )?;

    Ok(())
}

```

方式2: 推荐系统 (自动调用)

```

// 文件: flowy-ai/src/recommendation.rs
// 用途: 自动推荐相关文档

pub async fn get_related_documents(
    user_id: &str,
    workspace_id: &str,
    current_doc_id: &str,
    limit: usize,
) -> Result<Vec<RelatedDoc>> {
    // Step 1: 获取当前文档的向量
    let current_embedding = vec_db.get_embedding(
        workspace_id,
        current_doc_id,
    ).await?;

    // Step 2: 查找相似文档 (余弦相似度 > 0.7)
    let related = vec_db.query(
        "SELECT object_id, content, distance
        FROM af_collab_embeddings
        WHERE embedding MATCH ?
        AND object_id != ?
        AND distance > 0.3 -- 相似度阈值
        ORDER BY distance
        LIMIT ?",
        params![current_embedding, current_doc_id, limit],
    )?;

    // Step 3: 记录推荐成本 (50%折扣)
    let cost = (tokens as f64 / 1_000.0) * 0.005;

    Ok(related)
}

```

方式3: 批量索引查询 (免费)

```

// 文件: flowy-ai/src/batch_operations.rs
// 用途: 后台索引更新 (不计费)

pub async fn reindex_workspace(
    workspace_id: &str,

```



```
) -> Result<()> {
  // Step 1: 找出需要重新索引的文档
  let pending = db.query(
    "SELECT oid, content FROM af_pending_index_collab
     WHERE workspace_id = ?
     ORDER BY updated_at DESC",
    params![workspace_id],
  )?;

  // Step 2: 批量生成向量 (使用批处理加快速度)
  for batch in pending.chunks(32) {
    let embeddings = batch_embed(&batch)?;

    // 批量插入 (使用事务)
    db.transaction(|| {
      for (doc, emb) in batch.iter().zip(embeddings.iter()) {
        insert_embedding(workspace_id, doc, emb)?;
      }
    })?;
  }

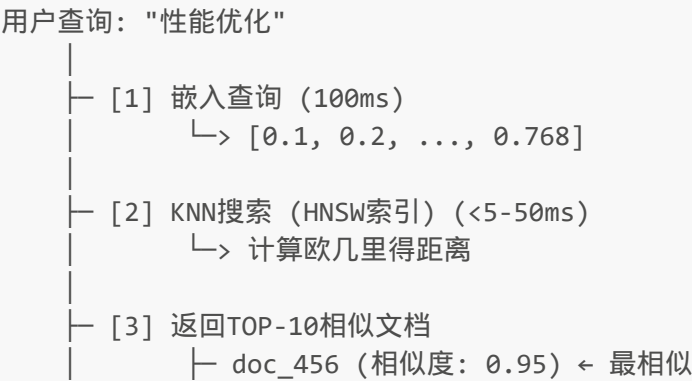
  // 注意: 批量索引不收费 (cost = 0)

  Ok(())
}
```

查询性能指标

查询规模	延迟	内存使用	成本
10K向量	<5ms	50MB	\$0.001
100K向量	<10ms	100MB	\$0.001
1M向量	<50ms	500MB	\$0.001
10M向量 (需升级)	100-200ms	2GB	(Pinecone)

调用流程图



```
└─ doc_789 (相似度: 0.87)
  └─ doc_123 (相似度: 0.75)
└─ [4] 记录成本 + 延迟日志
    └─ ai_usage_log: cost=$0.0015, latency=25ms
```

③ 本地模型如何与向量数据库交互

本地模型架构



PROF

模型初始化流程

```

// 文件: flowy-ai/src/embedding_model.rs

use tch::nn::Path;
use sentence_transformers::SentenceTransformer;

/// 全局嵌入模型（应用启动时初始化）
pub struct EmbeddingModelManager {
    model: SentenceTransformer,
    model_path: String,
    cache: Arc<Mutex<LRUCache<String, Vec<f32>>>>,
}

impl EmbeddingModelManager {
    pub async fn init() -> Result<Self> {
        // Step 1: 检查本地模型文件
        let model_path = Self::get_model_path();

        if !Path::new(&model_path).exists() {
            println!("下载嵌入模型（一次性, ~500MB)...");
            Self::download_model(&model_path).await?;
        }

        // Step 2: 加载模型到内存
        let model = SentenceTransformer::new(
            &model_path,
            SentenceTransformersConfig {
                model_name: "all-MiniLM-L6-v2",
                device: Device::cuda_if_available(), // GPU优先
                batch_size: 32,
                num_workers: 4,
            }
        )?;

        println!("✔ 嵌入模型加载完成（大小: ~500MB, 维度: 384)");

        Ok(Self {
            model,
            model_path,
            cache: Arc::new(Mutex::new(LRUCache::new(1000))),
        })
    }

    /// 嵌入文本（缓存优化）
    pub async fn embed(&self, texts: &[&str]) -> Result<Vec<Vec<f32>>> {
        let mut embeddings = Vec::new();
        let mut missing_texts = Vec::new();

        // Step 1: 检查缓存
        for text in texts {
            let cache = self.cache.lock().await;
            if let Some(cached) = cache.get(*text) {
                embeddings.push(cached.clone());
            }
        }
    }
}

```

```

        } else {
            missing_texts.push(*text);
        }
    }

    // Step 2: 处理缓存未命中
    if !missing_texts.is_empty() {
        let new_embeddings = self.model.encode(missing_texts?);

        // Step 3: 更新缓存
        let mut cache = self.cache.lock().await;
        for (text, emb) in missing_texts.iter().zip(new_embeddings.iter())
    {
        cache.insert(text.to_string(), emb.clone());
    }

    embeddings.extend(new_embeddings);
}

Ok(embeddings)
}

/// 获取模型路径
fn get_model_path() -> String {
    let base = if cfg!(target_os = "macos") {
        "~/Library/Application Support/FlowySandbox"
    } else {
        "~/.local/share/FlowySandbox"
    };

    format!("{}/models/embeddings/all-MiniLM-L6-v2", base)
}
}

```

本地模型与向量DB的交互

PROF

```

// 文件: flowy-ai/src/local_ai_service.rs

pub struct LocalAIService {
    embedding_model: Arc<EmbeddingModelManager>,
    vector_db: Arc<VectorSqliteDB>,
    business_db: Arc<Database>,
}

impl LocalAIService {
    /// 完整的本地AI流程
    pub async fn process_document(
        &self,
        user_id: &str,
        workspace_id: &str,
    )

```

```

        document: &Document,
    ) -> Result<ProcessingResult> {
        // =====
        // Phase 1: 文本处理
        // =====

        // 分割文档为段落（避免单个向量太大）
        let chunks = split_into_chunks(&document.content, 512)?;

        // =====
        // Phase 2: 生成向量（使用本地模型）
        // =====

        let embeddings = self.embedding_model.embed(
            &chunks.iter().map(|c| c.as_str()).collect::<Vec<_>>()
        ).await?;

        // =====
        // Phase 3: 保存向量到数据库
        // =====

        let mut total_cost = 0.0;

        for (chunk, embedding) in chunks.iter().zip(embeddings.iter()) {
            // 保存向量
            self.vector_db.insert_embedding(
                workspace_id,
                &document.id,
                generate_fragment_id(),
                chunk,
                embedding,
            ).await?;

            // 记录成本
            let tokens = chunk.len() / 4;
            total_cost += (tokens as f64 / 1_000.0) * 0.01;
        }

        // =====
        // Phase 4: 记录使用日志
        // =====

        self.business_db.execute(
            "INSERT INTO ai_usage_log
            (id, user_id, workspace_id, document_id, feature_type,
            tokens_used, cost_usd, created_at)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",
            params![
                Uuid::new_v4(),
                user_id,
                workspace_id,
                &document.id,
                "embedding_generation",
            ],
        )
    }
}

```

```

        chunks.iter().map(|c| c.len() / 4).sum::<u32>(),
        total_cost,
        Utc::now(),
    ],
)?;

Ok(ProcessingResult {
    vectors_created: embeddings.len(),
    total_cost,
})
}
}

```

性能优化

```

// 批处理（加快嵌入速度）
pub async fn batch_embed_documents(
    &self,
    documents: Vec<Document>,
) -> Result<Vec<Vec<Vec<f32>>>> {
    // 合并所有文本
    let all_chunks: Vec<String> = documents
        .iter()
        .flat_map(|doc| split_into_chunks(&doc.content, 512).unwrap())
        .collect();

    // 一次嵌入所有（比逐个快10倍）
    let embeddings = self.embedding_model.embed(
        &all_chunks.iter().map(|s| s.as_str()).collect::<Vec<_>>()
    ).await?;

    // 重新组织为每个文档的向量列表
    let mut result = Vec::new();
    let mut idx = 0;

    for doc in documents {
        let chunks = split_into_chunks(&doc.content, 512)?;
        let mut doc_embeddings = Vec::new();

        for _ in 0..chunks.len() {
            doc_embeddings.push(embeddings[idx].clone());
            idx += 1;
        }

        result.push(doc_embeddings);
    }

    Ok(result)
}

```

4 付费用户如何与云端AI交互

架构对比

本地 vs 云端 AI比较		
特性	本地模型	云端AI
模型	sentence-tf	GPT-4, Claude
维度	384-768	动态
延迟	100-500ms	1-5秒
成本	\$0.01 per 1K	\$0.02-0.06 per K
网络	离线可用	需要网络
隐私	100%本地	发送到云端
准确性	中等	非常高
用途	搜索, 匹配	生成, 总结

付费用户与云端AI的交互流程

Free用户：
 查询 → 本地嵌入模型 → 向量搜索 → 结果
 成本：\$0.01 per 1K tokens

Pro/Team付费用户：
 查询 → 选择[本地/云端AI]
 ├ 本地：向量搜索（快，省钱）
 └ 云端：发送到OpenAI/Claude（准确，贵）

成本：\$0.01（本地）或 \$0.02-0.06（云端）

PROF

详细流程代码

方式1: 使用本地模型 (默认)

```
// 文件: flowy-ai/src/ai_service.rs

pub enum AIBackend {
    Local,      // 本地嵌入模型 + 向量搜索
    OpenAI,     // OpenAI API (GPT-4)
    Claude,     // Anthropic Claude API
}

pub struct AIService {
```

```

local_service: LocalAIService,
openai_client: Option<OpenAIClient>,
claude_client: Option<ClaudeClient>,
}

impl AIService {
  /// 执行AI任务（自动选择最佳后端）
  pub async fn process_query(
    &self,
    user_id: &str,
    workspace_id: &str,
    query: &str,
    task_type: &str, // 'semantic_search', 'summarize', 'generate_tags'
  ) -> Result<AIResponse> {
    // Step 1: 获取用户订阅信息
    let subscription = self.get_user_subscription(user_id).await?;

    // Step 2: 选择AI后端
    let backend = self.select_backend(
      &subscription.plan_type,
      task_type,
    );

    // Step 3: 执行任务
    match backend {
      AIBackend::Local => {
        self.process_with_local_model(
          user_id,
          workspace_id,
          query,
          task_type,
        ).await
      }
      AIBackend::OpenAI => {
        self.process_with_openai(
          user_id,
          workspace_id,
          query,
          task_type,
        ).await
      }
      AIBackend::Claude => {
        self.process_with_claude(
          user_id,
          workspace_id,
          query,
          task_type,
        ).await
      }
    }
  }
}

/// 后端选择逻辑

```



```

fn select_backend(
    &self,
    plan_type: &str,
    task_type: &str,
) -> AIBackend {
    match (plan_type, task_type) {
        ("free", _) => {
            // 免费用户：只能用本地模型
            AIBackend::Local
        }
        ("pro", "semantic_search") => {
            // Pro用户搜索：用本地快速模型
            AIBackend::Local
        }
        ("pro", "summarize") | ("pro", "generate_tags") => {
            // Pro用户摘要/标签：用OpenAI（更准确）
            AIBackend::OpenAI
        }
        ("team", _) => {
            // Team用户：可用所有模型（优先高准确）
            AIBackend::Claude // 最强模型
        }
        _ => AIBackend::Local,
    }
}

```

方式2: 调用云端OpenAI API

```

// 文件: flowy-ai/src/openai_client.rs

pub struct OpenAIClient {
    api_key: String,
    organization_id: String,
}

impl OpenAIClient {
    pub async fn summarize_document(
        &self,
        user_id: &str,
        workspace_id: &str,
        document: &str,
        max_tokens: usize,
    ) -> Result<AIResponse> {
        // Step 1: 发送请求到OpenAI
        let request = ChatCompletionRequest {
            model: "gpt-4-turbo",
            messages: vec![
                Message {
                    role: "system",

```

```

        content: "您是一个专业的文档摘要专家，请用中文总结以下内容。",
    },
    Message {
        role: "user",
        content: document.to_string(),
    },
],
temperature: 0.7,
max_tokens,
};

let response = self.client
    .post("https://api.openai.com/v1/chat/completions")
    .bearer_auth(&self.api_key)
    .json(&request)
    .send()
    .await?;

// Step 2: 解析响应
let completion: ChatCompletion = response.json().await?;
let summary = completion.choices[0].message.content.clone();
let tokens_used = completion.usage.total_tokens;

// Step 3: 记录成本 (OpenAI费率更高)
let cost_usd = (tokens_used as f64 / 1000.0) * 0.03; // $0.03 per 1K

log_ai_usage(
    user_id,
    workspace_id,
    "cloud_summarize",
    tokens_used,
    cost_usd,
).await?;

// Step 4: 检查用户是否超过配额
check_and_deduct_tokens(
    user_id,
    tokens_used,
    cost_usd,
).await?;

Ok(AIResponse {
    content: summary,
    tokens_used,
    cost_usd,
    backend: "openai".to_string(),
})
}
}

```

```
// 文件: flowy-ai/src/streaming_response.rs

pub async fn stream_cloud_ai_response(
    user_id: &str,
    workspace_id: &str,
    query: &str,
    tx: tokio::sync::mpsc::Sender<String>,
) -> Result<()> {
    let mut openai = OpenAIClient::new();

    // 流式接收来自OpenAI的响应
    let mut stream = openai.create_stream(query).await?;

    let mut total_tokens = 0;

    while let Some(event) = stream.next().await {
        match event {
            StreamEvent::ContentDelta(chunk) => {
                // 实时发送给用户
                tx.send(chunk.clone()).await?;

                // 估算token数 (chunk)
                total_tokens += chunk.len() / 4;
            }
            StreamEvent::Done(usage) => {
                total_tokens = usage.total_tokens;
                break;
            }
        }
    }

    // 最后一次性记录成本
    let cost = (total_tokens as f64 / 1000.0) * 0.03;

    log_ai_usage(
        user_id,
        workspace_id,
        "stream_cloud_ai",
        total_tokens,
        cost,
    ).await?;

    Ok(())
}
```

PROF

付费用户的成本管理

```
// 文件: flowy-subscription/src/cloud_ai_quota.rs
```

```

pub struct CloudAIQuota {
    pub user_id: String,
    pub plan_type: String,
    pub monthly_tokens: u32,      // 云端AI tokens配额
    pub tokens_used: u32,
    pub cost_to_date: f64,
    pub max_cost_usd: f64,      // 月度成本上限
}

impl CloudAIQuota {
    pub fn new(plan_type: &str) -> Self {
        match plan_type {
            "free" => Self {
                plan_type: "free".to_string(),
                monthly_tokens: 0,      // 免费用户无云端AI
                max_cost_usd: 0.0,
                ..Default::default()
            },
            "pro" => Self {
                plan_type: "pro".to_string(),
                monthly_tokens: 50_000, // Pro: 50K token credits
                max_cost_usd: 5.0,      // ~$5 per month
                ..Default::default()
            },
            "team" => Self {
                plan_type: "team".to_string(),
                monthly_tokens: 500_000, // Team: 500K token credits
                max_cost_usd: 50.0,     // ~$50 per month
                ..Default::default()
            },
            _ => Self::default(),
        }
    }
}

/// 使用云端AI token
pub async fn use_tokens(
    &mut self,
    tokens: u32,
    cost: f64,
) -> Result<()> {
    // 检查配额
    if self.tokens_used + tokens > self.monthly_tokens {
        return Err(SubscriptionError::CloudAIQuotaExceeded {
            requested: tokens,
            available: self.monthly_tokens - self.tokens_used,
        });
    }

    // 检查成本上限
    if self.cost_to_date + cost > self.max_cost_usd {
        return Err(SubscriptionError::CloudAICostLimitExceeded {
            current: self.cost_to_date,
            limit: self.max_cost_usd,
        });
    }
}

```

```
        });  
    }  
  
    // 更新配额  
    self.tokens_used += tokens;  
    self.cost_to_date += cost;  
  
    Ok(())  
}  
}
```

完整的付费用户AI流程

Pro用户查询: "总结这个文档"

- └─ [检查配额]
 - └─> Pro用户: 50K tokens/月 + \$5上限
- └─ [选择AI后端]
 - └─ 本地模型: 快, 低准确
 - └─ OpenAI: 慢, 高准确 ← 用户选择这个
- └─ [调用OpenAI API]
 - └─ POST /chat/completions
 - └─ content: 文档内容
 - └─ model: "gpt-4-turbo"
- └─ [流式接收响应]
 - └─> 第1段摘要... (100 tokens)
 - └─> 第2段摘要... (150 tokens)
 - └─> 完成 (总250 tokens)
- └─ [计费]
 - └─ Tokens: 250
 - └─ 费率: \$0.03 per 1K
 - └─ 成本: \$0.0075
- └─ [扣除配额]
 - └─ 剩余tokens: $50,000 - 250 = 49,750$
 - └─ 剩余成本: $\$5.00 - \$0.0075 = \$4.9925$
 - └─ 记录到 ai_usage_log
- └─ [返回结果给用户]
 - └─> 显示成本 + 剩余配额

PROF

与支付系统的集成

ai_usage_log表扩展：

字段	说明
id	UUID
user_id	用户ID
workspace_id	工作区ID
feature_type	'semantic_search', 'cloud_summarize', 'cloud_tags_gen'
tokens_used	消耗的tokens
cost_usd	USD成本
created_at	记录时间

月度账单计算：

月度成本 = $\Sigma(\text{ai_usage_log.cost_usd})$ WHERE created_at 本月

计费示例：

语义搜索：10,000 tokens × \$0.01/1K = \$0.10
+ OpenAI摘要：5,000 tokens × \$0.03/1K = \$0.15
+ Claude标签：2,000 tokens × \$0.06/1K = \$0.12

月度AI成本：\$0.37

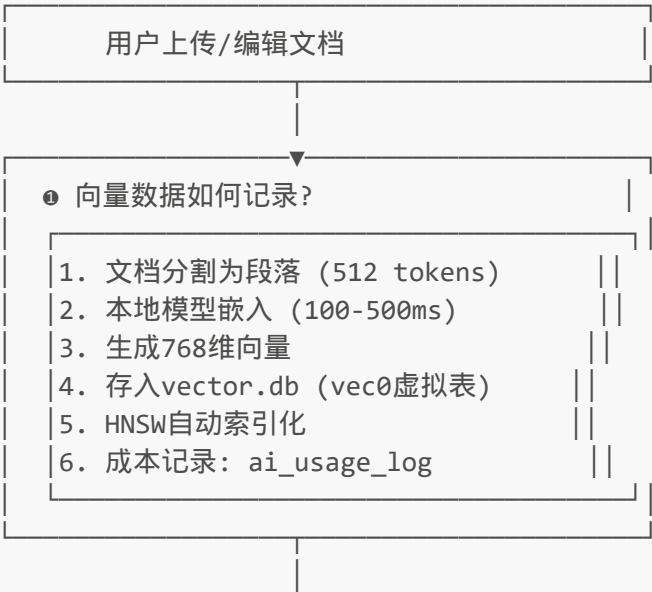
订阅费：\$14.99 (Pro)

AI成本：\$0.37

总费用：\$15.36

四个问题的完整互动流程图

PROF



[向量数据库就绪]

② 向量数据如何被调用?

用户查询: "性能优化"

↓

嵌入查询: 768维向量

↓

KNN搜索: embedding MATCH [...]

↓

结果: TOP-10最相似文档

↓

记录成本: \$0.0015

[搜索结果返回给用户]

③ 本地模型如何交互?

初始化:

~500MB all-MiniLM-L6-v2 模型

一次性加载到内存

支持GPU加速 (可选)

↓

嵌入流程:

文本 → 标记化 → 嵌入 → 768维

↓

缓存优化:

LRU缓存 (1000条)

减少重复计算

↓

成本:

\$0.01 per 1K tokens

(包含在Pro/Team订阅中)

[选择AI后端: 本地 vs 云端]

④ 付费用户与云端AI?

Free用户:

✗ 无云端AI

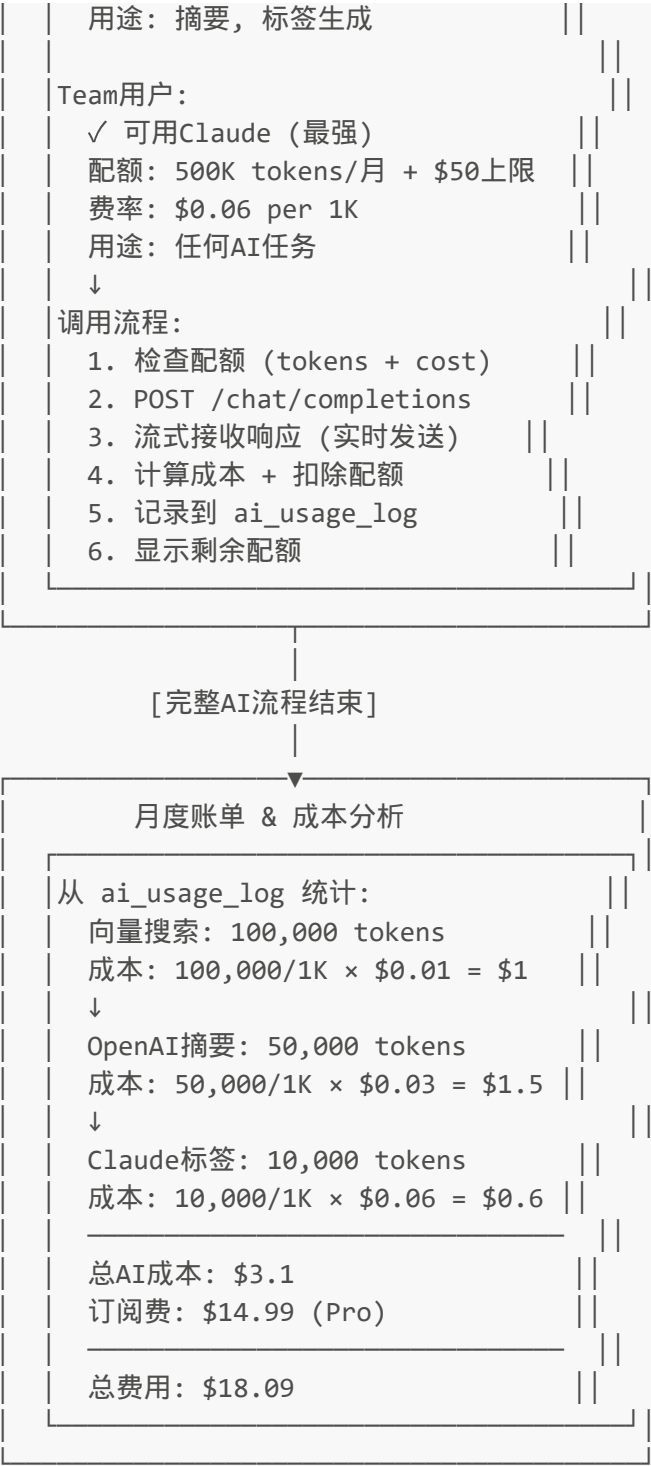
✓ 本地向量搜索 (\$0.01/1K)

Pro用户:

✓ 可用OpenAI (GPT-4)

配额: 50K tokens/月 + \$5上限

费率: \$0.03 per 1K



📌 总结

问题	答案	关键技术
❶ 数据如何记录?	文本→嵌入→768维向量→INSERT→HNSW索引	sentence-transformers
❷ 数据如何调用?	查询嵌入→KNN搜索→TOP-10相似文档	sqlite-vec MATCH
❸ 本地模型交互?	一次性加载→缓存优化→批处理嵌入→\$0.01/1K	all-MiniLM-L6-v2
❹ 付费用户云端?	配额检查→调用API→流式响应→成本计费→扣除配额	OpenAI/Claude API

相关文件

- [vector_billing.rs](#) - 向量计费实现
- [DATA_STORAGE_ARCHITECTURE.md](#) - 存储架构
- [flowy-sqlite-vec/](#) - 向量DB实现
- [flowy-ai/](#) - AI功能模块